# HIC Project 1 Report

Tatiana Tkacheva, Andrew Roddy, Savar Shrestha, Michael Bonanno
October 15, 2026

## Introduction

The app that we created is Bagl. This is an application that lets users purchase bagels and earn rewards! It also features hotkeys and an order history page so user's and view their previous orders.

We created an application with the goal of fully implementing Schneiderman's 8 golden rules.

## 8 Golden Rules

Schneiderman's 8 golden rules are:

1. Strive for Consistency
2. Enable Frequent Users to Use Shortcuts
3. Offer Informative Feedback
4. Design Dialogue to Yield Closure
5. Offer Simple Error Handling
6. Permit Easy Reversal of Actions
7. Support Internal Locus of Control
8. Reduce Short Term Memory Load

## Best Implementation

The page or feature that best implements each golden rule.

1. All pages, as the design is consistent across the application
2. The Keyboard Shortcuts popup activated when the user presses 'Alt/Option'
3. Order Page
4. Checkout Page
5. Rewards Page
6. Cart Page
7. Cart Page
8. History Page

## Home Page




Figure 1: The home page.

When the user first opens the application they are taken to the home page. This page shows the user our rewards and explains the website. It also features buttons that suggest the next actions the user will likely take. The buttons allow the user to quickly place an order or become a member.

The entire application applies the first golden rule of consistency. All of the applications have the same navbar, footer, and color pallette. They share the same accent color of `#ff922f` orange, our company color. On this home page, the two buttons at the bottom allow the user to be fully in control applying by allowing them to easily choose to either order now or join the rewards program, applying the seventh golden rule.

## Order Page




Figure 2: The order page.

The order pag shows all of our different bagel options. The user can easily click on any of the bagels to add one of them to their cart. Once they click on one of the bagels, they are taken to the Bagl Info page to increase or decrease their bagel count.

This page implementes golden rule eight as it has the bagel image, name, and price on every card, reducing the short term memory load needed by the user. It also pops out each bagel, and highlights on hover, as informative feedback for the user implementing golden rule three.

## Bagl Info Page

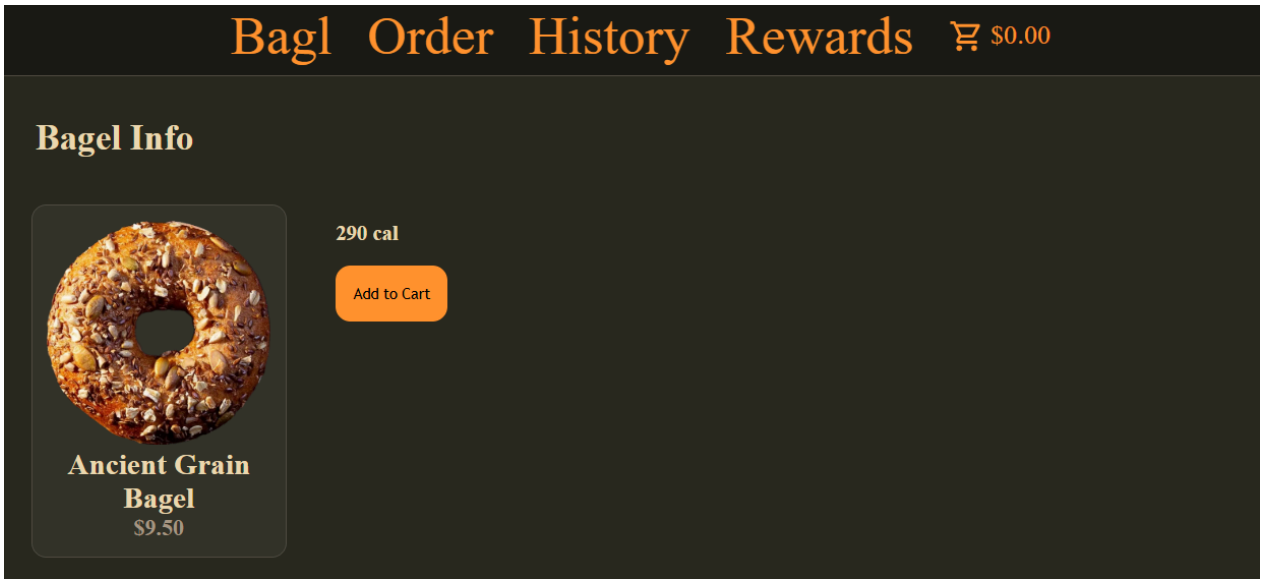



Figure 3: The bagel info page.

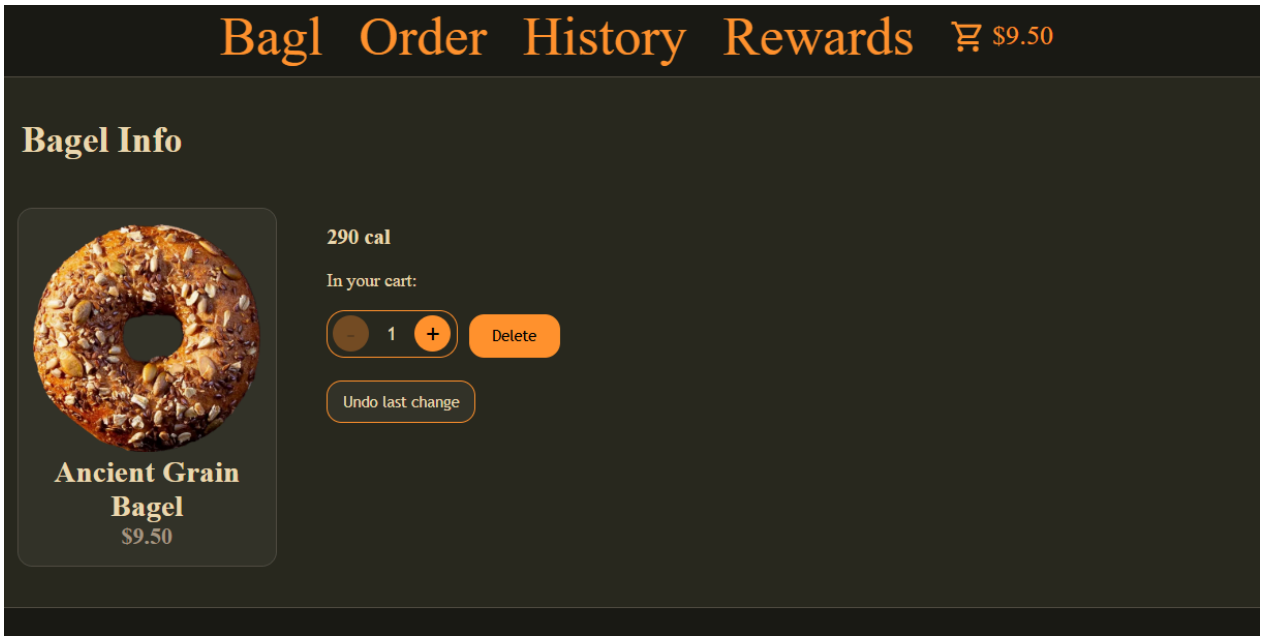



Figure 4: The bagel info page after pressing 'Add to Cart'.

After clicking on a bagel the user is taken to the Bagl Info Page. This allows the user to see the calorie information relating to each bagel and allows them to confirm that they would like to add the bagel to the cart. Once the bagel is added to the cart the user can then increase or decrease the number of bagels that they would like to purchase. If they decide against purchasing the bagel they can easily press the `Delete` button. The cost in the cart also updates live as the number of bagels in the cart increases and decreases.

This page offers informative feedback as pressing add to cart quickly becomes the quantity controls. The cost of the entire purchase also updating in the navbar as the user adds and subtracts amounts adds to this informative feedback implementing the third golden rule. We offer simple error handling as we disable the minus button at 1 bagel. This prevents the user from having 0 or negative bagels, implementing rule five. We also allow the user to undo their actions as `Delete` removes the bagel from the cart and the minus button allows the user to undo accidentally adding an extra bagel. This implements rule six. We also reduce short term memory load as the user can see exactly how many of that bagel are in their cart in between the plus and minus, and the total cost of the transaction easily in the top right. This reduction of short term memory load helps to implement rule eight.

## Cart Page




Figure 5: The cart page.

The cart page allows the user to see what bagels they are preparing to purchase. It allows us as users to increase and decrease count of each bagel. The checkout page also has a `Checkout` button that when clicked makes the purchase. When the user has rewards saved up, they can spend these rewards also on the checkout page by checking a box to confirm that they want to use the reward to get a free bagel.

The order summary always shows the item count and the subtotal which reduces the short term memory load of the user allowing them to always know exactly how much the purchase will cost them even as they add or remove bagels, implementing rule eight. The checkout is disabled when the cart is empty and a message is shown that tells the user their cart is empty. This helps to prevent errors before they happen and implements rule five. We are also able to easily increase the counts of our items and derease them easily, like the bagel info page, allowing easy reversal of actions, implementing rule six. By allowing the user to easily decide when to checkout, modify bagel counts, and have their total on the side, we make the user feel like they are completely in control implementing rule seven. By having the same quantity controls as the bagel information page we establish consistency across the application.

## Checkout Page

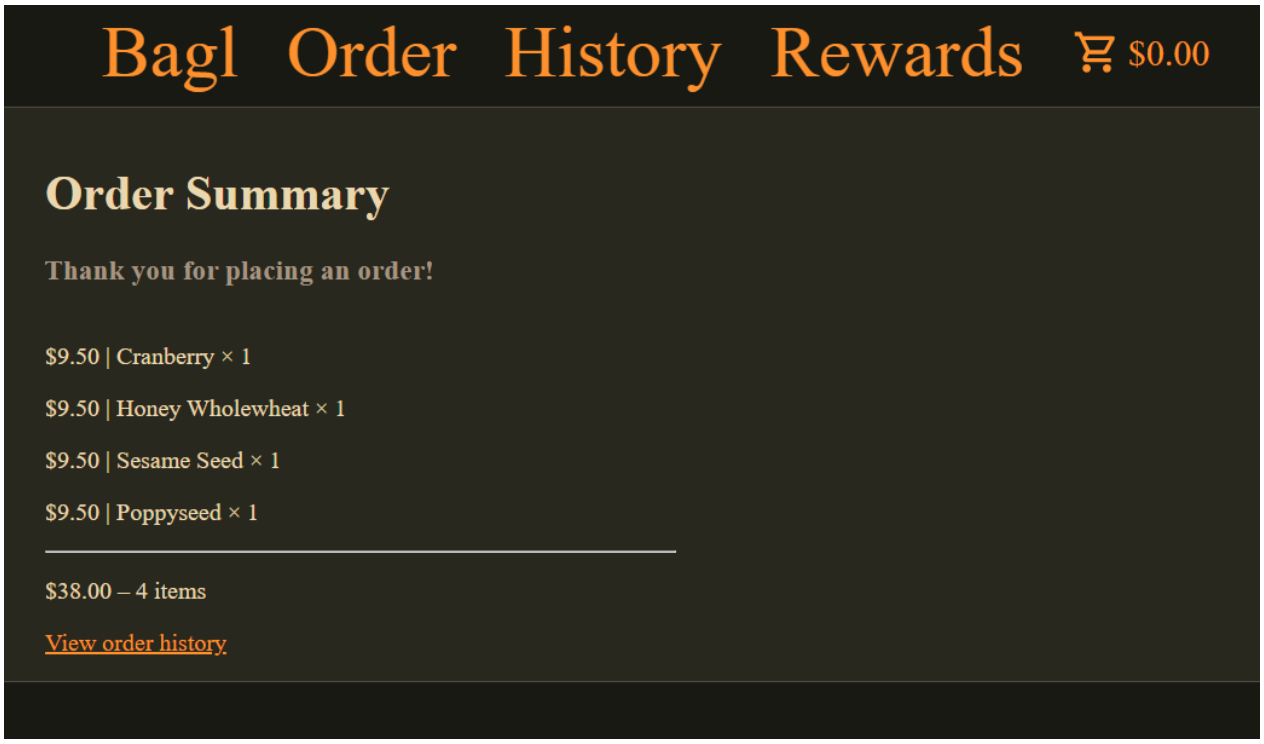



Figure 6: The checkout page.

... The checkout page shows the order summary after the user places their order. It shows a receipt like structure of the transaction the user just completed. It shows price of each item and the total cost. It also confirms that the user's order went through and links to the order history page at the bottom. if the user's cart is empty it redirects them back to the ordering page to get more bagels.

The checkout page has a thank you message and itemized receipt after completing the purchase. This dialogue yields closure to the user that their purchase went through implementing golden rule four. There is also a checking out message while it checks out the user. This is informative feedback to the user implementing golden rule three. The page offering a redirect to the order page to users who stumble across the checkout page with an empty cart offers simple error handling, implementing rule five.

## History Page

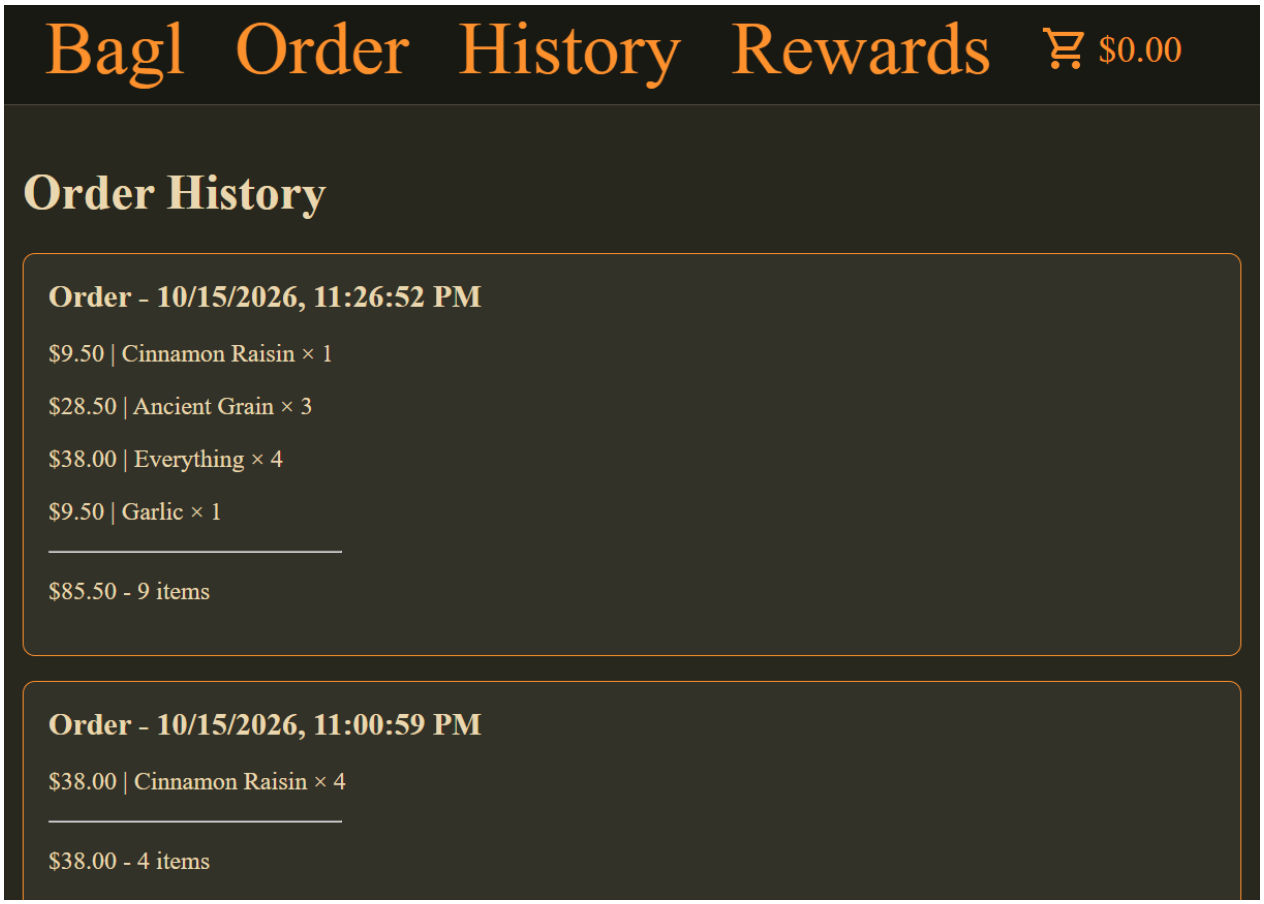



Figure 7: The history page.

The history page allows the user to see their entire history of every bagel they have purchased. It functions like a list of receipts, showing the user the time, date, prices, and total cost of each order.

This page allows the user to not have to remember all of their past purchases implementing rule eight. This not only reduces the user's short term memory load but also the long term memory load. Because completed orders are saved permanently, the user is able to check to confirm at any point that their order went through yielding closure and implementing rule four.

## Rewards Page

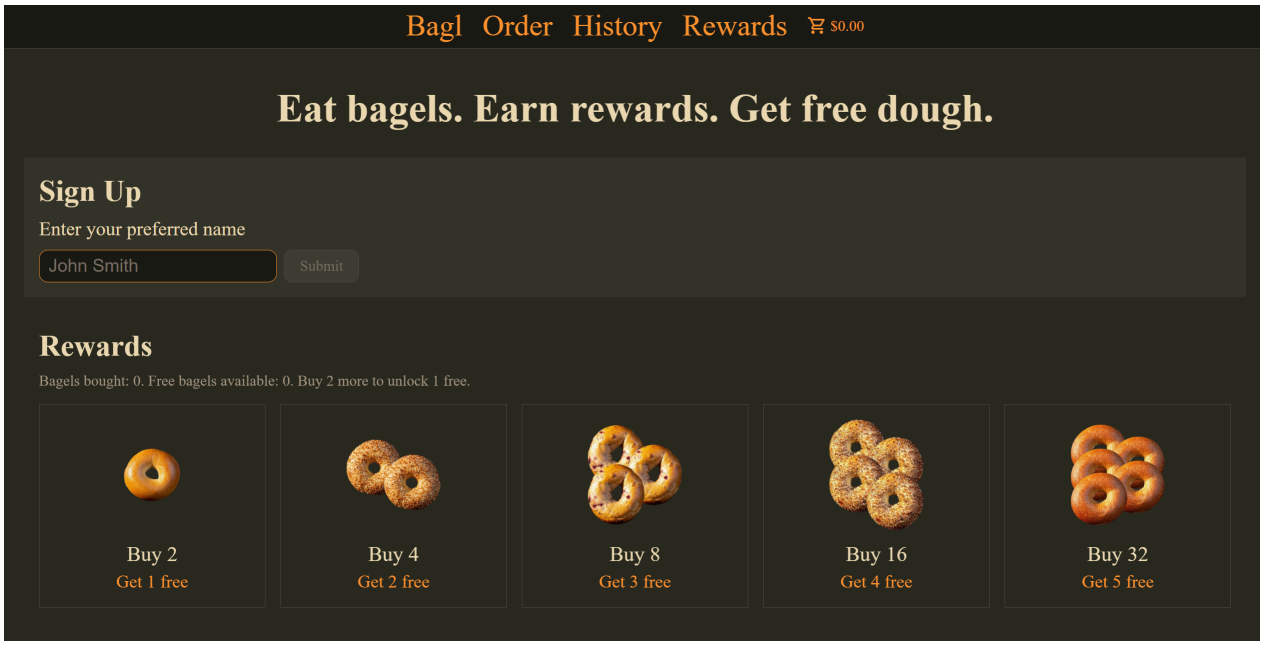



Figure 8: The rewards page.

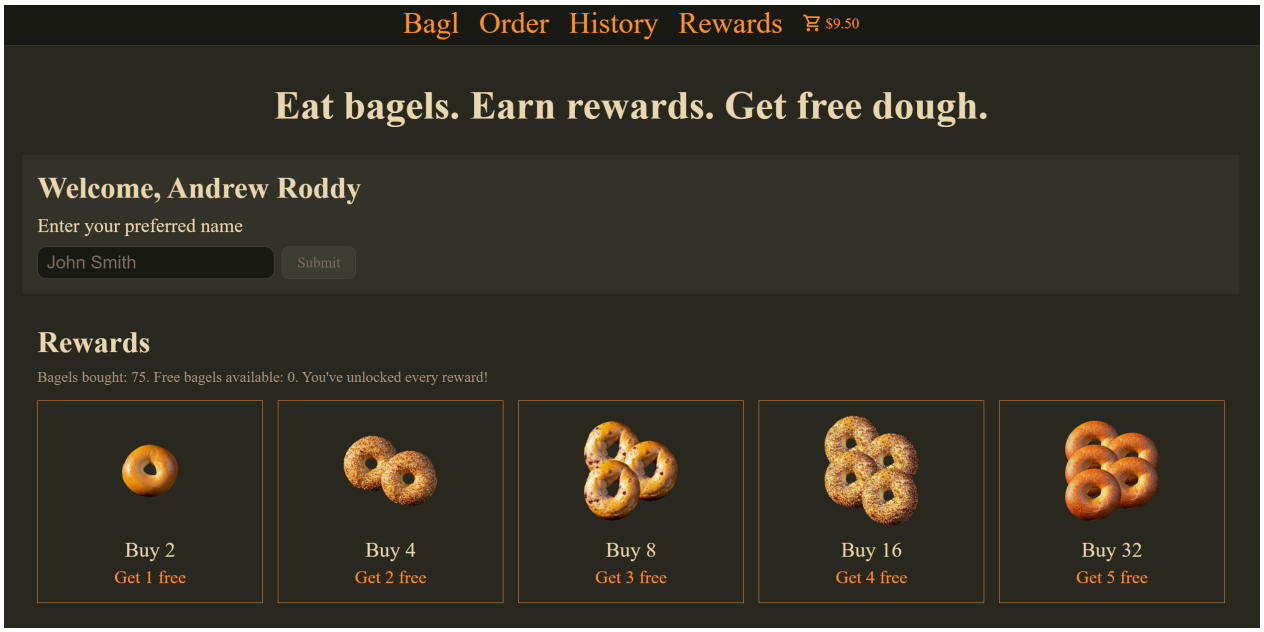



Figure 9: The rewards page with rewards unlocked.

The rewards page lets customers join the Bagl membership program by entering their preferred name. The input is sanitized and accepts only letters, spaces, hyphens and apostrophes and is saved in the browser's local storage. This makes it so that the home page can greet returning users by their name. Buying a bagel earns one point which is automatically added during checkout. The highest tier that you have reached determines how many free bagels a customer has earned in total. The page shows the customer's bagels bought, free bagels available, and how many more purchases unlock the next tier, with unlocked tier cards highlighted. In the cart when you have free bagels available a checkbox

appears, which the customer can select to get free bagels depending on their availability. All reward data is in local storage, it is tied to one browser and device.

## About Page




Figure 10: The about page.




Figure 11: The about page button in the footer.

The about page includes the story of how the business came to be about. It has the name of the individuals who contributed to make the business.

## Hotkeys




Figure 12: The hotkeys dropdown acccessed by pressing ? / Alt / Option.

At the bottom of the page, in the footer, the user is shown that if they press 'Alt', 'Option', or '?' it will display keyboard shortcuts. Users who use the application and explore it often will see this message. Power uses are used to pressing the Alt key in applications like Photoshop, Premiere Pro, or Word to view the list of available hotkeys. This parallel is also made for Mac users who are used to pressing the '?' key on websites. This allows power users to quickly and easily find the hotkeys. These hotkeys act as shortcuts to easily

The hotkeys feature allows the use of shortcuts for expert users, allowing them to faster navigate through the website, implementing rule two. The shortcuts are also ignored when users are holding control and command allowing them to still use the regular browser hotkeys. The user is also able to use the '?', 'Alt', and 'Option' keys to show or hide the overlay. If the user chooses to keep the overlay, it will stay on their screen for as long as they want. This supports the user's internal locaus of control and implements rule seven.

## Group Contributions

Andrew Roddy created the repository and initial README. He setup the structure of the SvelteKit project and organized the routes to navigate through the website. Then, he added all 11 of the bagel images that you see in the repository, made and styled the bagel card components in the order page. He continued to improve the CSS for the order page by adding effects to the card. He built the shopping cart application with a working total, and saved the cart to local storage and created a global color theme to have consistent looking pages. He also made shortcuts for the application to make it more accessible to a variety of users, and fixed hotkey bugs like shortcut firing while typing in input field, binding the option key as an alternative to alt for mac users, and wrote the installation instructions and created the report for submission.

Savar Shrestha created the discord chat and organized the group to have a total of 4 members. Initially he tested out the javascript framework by loading bagel images. Created the very first version of the rewards page and fixed color scheme issues. Redesigned the rewards page to have a "Sign Up" form and sanitized the input. Created the tier logic and cards with stacked bagel images. Built the reward point system which saves the name and points in local storage and connected it to the cart, home page, checkout, and order history, so the home page can greet returning users by name. He also fixed the font

size of different pages making them more visible. At the end, he co-authored the report with Andrew Roddy.

Tatiana Tkacheva built the bagel info page that adds bagels to the cart. She worked on the cart page and improved the interaction when clicking a bagel. She made the final structure for the cart page and cleaned up how the cart state is managed. She built the checkout page with confirmation and receipt of the purchase. Added a link to the order page when the customer goes to an empty cart. Afterwards, built the order history page and fixed a pricing bug. Also, redisgned the homepage layout.

Michael Bonanno created the home page that has a welcome heading, the mission statement for the business, the membership perks, and the buttons which lead you to the associated pages. Then, he aligned and adjusted the home page and created an about page writing about the history of the business.

## Running the application

You can check out the application at: https://bagl-pi.vercel.app/

If you would like to run the application locally, First run

```
npm install
```

Then to run the application use

```
npm run dev
```